manners depending on where those variables are declared and used. However, if we attempt to access a variable outside of its scope, we will receive another one of those friendly reminders in the form of a compiler error letting us know where we went wrong.

Because writing code for Arduino C is a little different from normal C source code, specifically in our use of the `setup()` function to perform one time only tasks, we will often use global variables to define pin numbers, set up thresholds, or define constants, so that all functions in a sketch can use the same values. This would not generally be the case with code written for other platforms, but it works for us.

Using Operators

Now that we have thoroughly explored what the variables are, what can we do with them? To answer this question we need to examine various **operators**, which are special characters used to perform specific operations. For now, we will stick with simple assignment, arithmetic, and compound operators, and get into some others in the next chapter.

When we talked about declaring variables, we used the **assignment operator**, `=`, which looks just like the equals sign but isn't exactly the equals sign we know and love from algebra class. Take a look at the following example:

```
myValue = 255;
```

Here, the assignment operator is telling the compiler to take the value or expression on the right side of the operator and assign that value to the variable on the left side of the `=` character. In this case, we are taking the value of 255 and putting it into a variable we are calling `myValue`. Considering the following hypothetical code fragment:

```
int myValue, newValue;
myValue = 255;
newValue = myValue;
```

The first line declares two integer type variables on the same line. (Yes, I know this is ninja sneaky, but as long as we separate the variable names with commas, we can declare multiple variables with the same data type.) The second line then assigns the numerical value `255` to the variable `myValue`. Then in the third line, remembering that assignments work somewhat counter intuitively right to left, we assign the value of `myValue` to the variable `newValue`. In this case `newValue` will now also hold the value of `255`. That should be crystal clear, right?

■ **Note** Because we can only declare a variable in the same scope once, we do not use the data type when assigning a value to a variable after the first declaration. If in the previous example we wrote `int myValue = 255;` instead of `myValue = 255;` we would have another compiler error.

Arithmetic Operators: +, -, *, /

With that out of the way, let's look at the simple arithmetic operators to add, subtract, multiply, and divide. To start with, let's say we wanted to add two numbers together and assign it to a variable; it would look something like the following: